

Birmingham City University

LOG REPORT

Data Analytics Pipeline: Implementation and Deployment

Lichess Open Database — MLOps pipeline implementation

Main body: 3610 words

CMP6230 — Coursework Task 2

Student: Nabin Oli

Student ID: 23189629

Academic Year 2024 / 25

Programme of study: BSc (Hons) Computer and Data Science

30th May 2026

Contents

1	Introduction	2
2	Shared utilities	2
3	Data ingestion	2
3.1	What	2
3.2	Why	3
3.3	How	3
4	Data pre-processing	5
4.1	What	5
4.2	Why	5
4.3	How	5
5	Model development	7
5.1	What	7
5.2	Why	7
5.3	How	7
6	Model deployment	8
6.1	What	8
6.2	Why	8
6.3	How	9
7	Model monitoring	10
7.1	What	10
7.2	Why	10
7.3	How	10
8	Reflective discussion	10
9	Conclusion	12
A	Code index	12
	References	14

1 Introduction

Coursework Task 1 for CMP6230 was a planning exercise where I compared candidate datasets, chose the Lichess Open Database (Lichess 2026; Creative Commons 2025), and sketched an MLOps pipeline with diagrams and storage justification. Task 2, documented here, is where that sketch became runnable software. This report is deliberately implementation-focused. I explain what I built, why I made particular storage and orchestration choices, and how each stage connects to code in the **LichessOps** GitHub repository 🐙 (Oli 2026f).

Lichess publishes a full monthly dump of rated games as a single compressed archive named in (`lichess-YYYY-MM.pgn.zst`) template format. The file is large, the schema is chess-specific (PGN headers rather than relational tables), and the licence is permissive to do further downstream Machine learning tasks. That combination is learning opportunity rich for Data Management and Machine learning Operations(MLOPS) practice.

My pipeline treats each new month as an incremental batch of data. Two audiences motivated the design from Task 1. First is analysts who want SQL and notebooks over curated tables, and second are automated ML lifecycle paths that retrains when fresh data lands. I adopted an **ELT** pipeline rather than classic ETL. Due to ELT use, Extraction and load happen first in our pipeline. The raw `.pgn.zst` data lands in MinIO Object store exactly in the format that lichess uploads on `database.lichess.org` and transformation runs later in Spark and DuckDB is the columnar database where our finally computed and columnar parquet formats dataset is stored.

2 Shared utilities

Before walking through ingestion or training, it is worth describing the small shared library I extracted once several packages started duplicating the same path logic. Early prototype version had several hard-coded directories after repetitive copy-paste, I centralised the behaviour under `libs/shared` making the code reusable across different components.

Each component has configuration loading. The main use of configuration loading is to (🐙Oli 2026b) walk up from the package until it finds `pyproject.toml`, then deep-merge `config/default.yaml` which a config file with a package-specific file. As a developer, This pattern eased up profoundly in configuration management and let me modify MinIO endpoints or DuckDB paths in one place during Docker bring-up without touching Python imports. None of these modules are glamorous, but they made the pipeline development more efficient and less error-prone.

3 Data ingestion

3.1 What

Data ingestion, in this project mainly involves pulling the monthly Lichess database dump from the public facing website `https://database.lichess.org` and storing it in durable object storage so every later stage uses the same bytes. The input is the template URL Lichess publishes from `database.lichess.org` where we download a Zstandard-compressed PGN file (Meta Platforms

2026) named `lichess-YYYY-MM.pgn.zst`. It is a compressed `.pgn` file which is a standard format to represent the chess game. Each game inside this `.pgn` file is text with headers (player names, ratings, opening, result). The output of ingestion is not yet a dataframe. it is that validated archive sitting in the `lichess-raw` bucket (MinIO, Inc. 2026) in a format which is ready for Spark for transformation.

The visible components are a Python downloader CLI, checksum validation, boto3 uploads, MinIO provisioned via Docker Compose (Oli 2026r), and Apache Airflow (Apache Software Foundation 2026a) as the scheduler here.

3.2 Why

I chose MinIO because PGN and `.zst` blobs do not map cleanly onto a columnar warehouse table. Object storage treats them as opaque files with cheap immutability and no semantics. I can reprocess April 2024 months later without re-downloading if the raw key still exists. Here, Checksum validation matters due to size of raw file. If the checksum fails then early detection prevents issues. Catching these issues early helps to save time and resources early in the pipeline.

Airflow runs the DAG on `0 3 1 * *` (03:00 on the first day of each month) so developer does not need to manually click download when Lichess typically publishes. ELT fits my development setup. Here transformation stays on Spark and DuckDB.

3.3 How

The `lichess_monthly_ingestion` DAG (Oli 2026n) shells out to `lichess-data`. That Keeps tasks testable from the terminal with the same flags Airflow passes.

On the default path (`use_elt=True`):

1. **download_shard**: `lichess-data download` calls `download_month` in (Oli 2026g). The client resumes partial files and can verify SHA256 against Lichess’s published sums while bytes still stream.
2. **upload_raw**: `upload_raw_shard` in (Oli 2026z) pushes to `s3://lichess-raw/` using (Oli 2026w). When `LICHESS_STORAGE_BACKEND=local`, upload is a no-op for laptop-only runs.
3. **validate_shard**: `validate_checksum` in (Oli 2026a) re-checks integrity. I treat a failure as a hard stop because the DAG should not proceed to Spark on a bad object.
4. Downstream tasks: (`spark_transform`, `duckdb_sync`) consume the raw key which they are preprocessing but depend on this stage finishing cleanly.

Bringing up storage uses the Compose `core` profile from the repo root `docker compose -profile core up -d` which also starts MinIO and a one-shot job that creates `lichess-raw`, `lichess-processed`, and `mlflow-artifacts`. Retrieval for later stages is literally an S3 GET on the month prefix which analysts or Spark both read the same URI via (Oli 2026y).

For retrieval, `raw_s3_uri(month)` returns the full `s3://` key, which DuckDB’s `httpfs` extension or Spark’s S3 configuration consumes. That indirection is how ingestion meets the brief’s “storage and retrieval” wording even before any ML features exist.

When Lichess rotates CDN behaviour, the downloader reads the monthly index rather than hard-coding URLs. That small robustness mattered because archive paths changed between

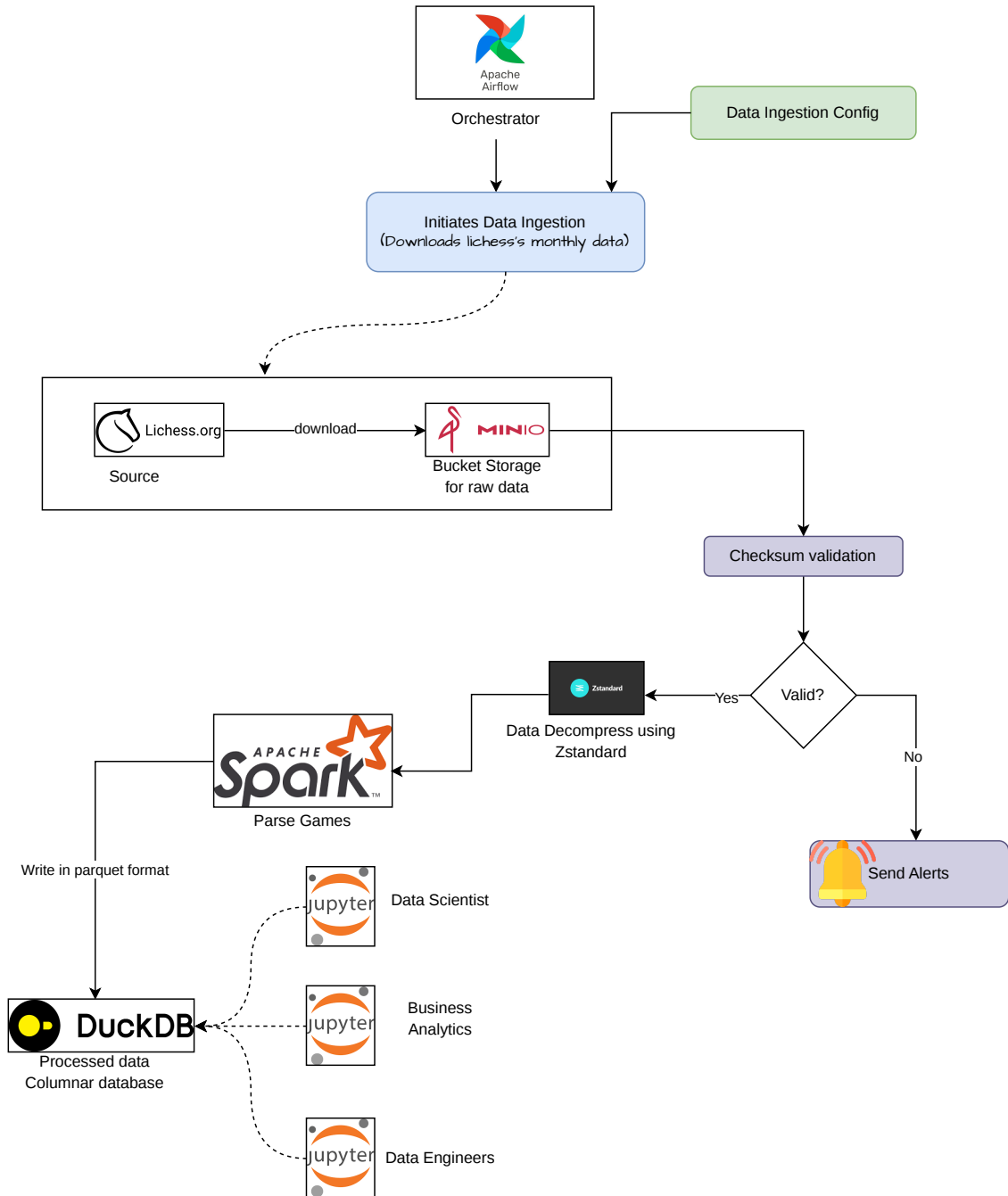


Figure 1: Data ingestion: download, checksum validation, and load to MinIO raw bucket.

academic years.

4 Data pre-processing

4.1 What

Pre-processing turns the chess games into tabular features which later ML libraries can ingest. Inputs are the raw `.pgn.zst` in MinIO. Outputs include partitioned Parquet under `lichess-processed`, a DuckDB file with star-schema tables which are Pandas-derived feature in Parquet format. Feast train/test datasets, and Great Expectations validation artefacts. If this stage is wrong, the model stage inherits silent bugs. Hence the emphasis has been given on validation before training.

4.2 Why

PGN is the lingua franca of chess software but a poor format for data analytics efficiency. It consists of variable header keys, multi-line movetext and comments which doesn't fit the tabular format required for efficient data analysis. We store the final artifact file as Apache Parquet (Apache Software Foundation 2026b) which provides us columnar compression and a schema which is compatible across Spark, DuckDB, and Pandas. Here, DuckDB lets run SQL checks such as (*e.g.* “how many games lack an ECO code?”). For feature storage, I employed Feast. Feast (Feast 2026) handles point-in-time joins so I do not leak future results into past rows, a mistake I almost made when I was first time computing rolling win rates. Great Expectations (Great Expectations 2026) encodes data validation expectations as a checkpoints. When a new monthly data arrives with a schema drift, then the pipeline halts the process instead of training on that drifted data.

4.3 How

We do following operations in the pre-processing stage:

Spark transform: Task `spark_transform` submits work in  Oli 2026x. The job streams the Zstandard payload, parses headers into structured fields, builds `fact_games` plus dimension tables, and writes a wide games table to the processed bucket. I use the `pipeline` Compose profile (`core + pipeline`) when RAM allows; otherwise `run_local_transform` keeps development unblocked.

DuckDB sync.  Oli 2026c registers Parquet from MinIO through `httpfs` and exports a single wide table for the Python feature code. Analysts can attach the same `.duckdb` file in the CLI container (`tools` profile) and run ad hoc SQL—that was the “interactive consumer” from Task 1.

Pandas pipeline. `preprocess_features` executes ten stages defined in  Oli 2026u via `PIPELINE_STAGES`: (1) `parse_event` maps Lichess event strings to Bullet/Blitz/Classical; (2) `encode_result` turns 1-0, 0-1, 1/2-1/2 into class labels; (3)–(4) date and clock features; (5) `impute_ratings` for missing values; (6) title flags; (7) opening and ECO family; (8) `add_historical_pregame_features` for rolling player form; (9) move-count summaries. I kept each step as a function so unit tests could target one transformation at a time.

DuckDB tables and Parquet more than necessary—Feast needs its registry, DuckDB needs SQL, sklearn needs a flat file.

Analytical processing angle. The brief asks how data supports ML and investigation. SQL answers distribution questions (rating histograms, opening popularity by month). The Pandas pipeline encodes domain knowledge (Bullet vs Blitz, title hierarchies). Feast enforces temporal discipline. Together they mean preprocessing is not a single script but a chain where each tool does one job well. When GE fails, I inspect which expectation broke—often a new Lichess header field appeared—and patch transforms before re-running the DAG.

5 Model development

5.1 What

Model development trains a supervised classifier to predict game outcome from pre-game information: ratings, titles, time control, opening family, and historical form. The business question from my draft notes was practical—if a player considers a given opening at their rating, what outcome distribution should they expect? Labels are three-way: white win, black win, draw (encoded numerically after `encode_result`). Deliverables include `model.joblib` on disk, metrics JSON, and a registered model in MLflow (LF AI & Data Foundation 2026).

5.2 Why

Game-level rows are the natural ingestion grain, but players think in “my colour, my rating band, my opening”. 🟡Oli 2026i expands each game to player-centric rows via `to_player_perspective` so the model answers that question. MLflow was already in the Task 1 diagram; using it for real gave me experiment IDs, parameter blobs, and a staging registry instead of folders named `model_final_v3_actually_final`.

5.3 How

Airflow task `train_model` calls `lichess-models train`. Inside 🟡Oli 2026m I wrap a scikit-learn (scikit-learn developers 2026) `Pipeline`: 🟡Oli 2026k builds a `ColumnTransformer` for numeric and categorical columns, then a classifier slot searched with `GridSearchCV` or `RandomizedSearchCV` and `TimeSeriesSplit` so validation respects time order. Candidate estimators include logistic regression, random forest, and histogram gradient boosting; grids live in `PARAM_GRIDS`.

The best pipeline serialises to `artifacts/lichess_models/{run_id}/model.joblib`. 🟡Oli 2026j scores the hold-out split and logs accuracy, macro-F1, and confusion-friendly counts to MLflow. 🟡Oli 2026l publishes under the name `lichess-outcome-predictor`. The MLflow stack 🟡(Oli 2026s) uses Postgres for metadata and MinIO (`mlflow-artifacts` bucket) for large artefacts—start `core` and `ml` profiles together or tracking breaks.

I also kept notebooks under `notebook/` for exploratory work; the CLI path is what Airflow relies on. Hyperparameter search is not cheap on a full month—during development I subsampled months or used the legacy extract path to iterate faster before letting the DAG run end-to-end.

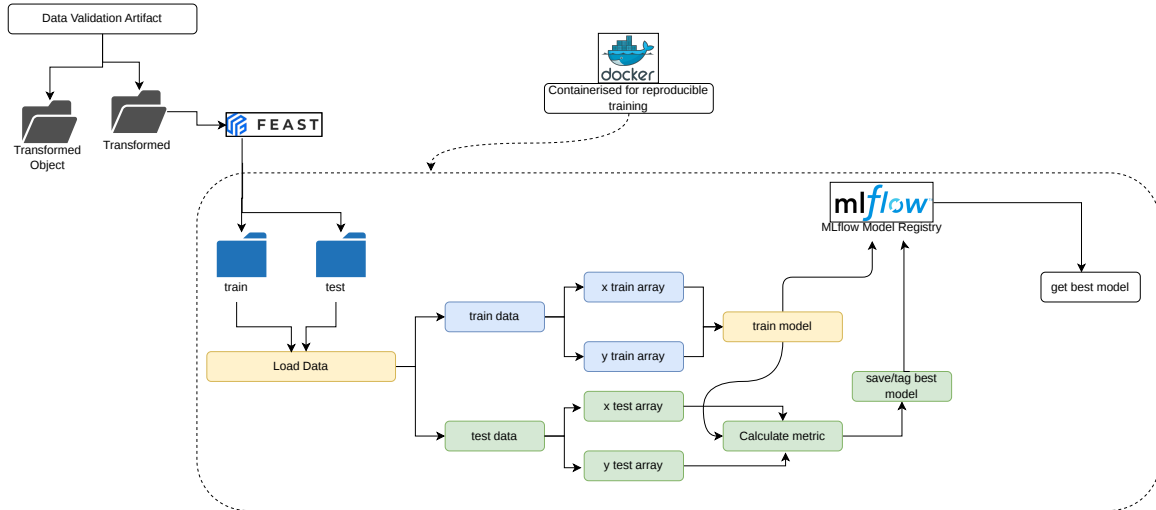


Figure 3: Model training: load splits, fit with hyperparameter search, log to MLflow.

The multi-class framing matches Lichess results better than treating draws as noise. I have not yet wired the opening weakness report (`packages/lichess_models/src/lichess_models/analyze.py`) into the monthly DAG; it remains a manual analysis script for now.

Metrics and reproducibility. Hold-out evaluation logs accuracy and macro-F1; I watch both because class imbalance makes accuracy optimistic. MLflow stores the conda-free environment implicitly through pip requirements in the repo; I still note the git commit hash manually in run tags when experimenting. For coursework evidence, the combination of `model.joblib`, MLflow UI screenshots, and cited training code satisfies “development of ML models” better than a single accuracy number in this PDF.

Retrieval of training data. Splits live under artefact paths returned by `split_paths(month)` in `Oli 2026i`. Airflow passes `-month` or `-previous-month` consistently across tasks so training never points at the wrong shard. That sounds obvious, but mismatched month parameters caused a silent train-on-January / evaluate-on-February bug during one all-nighter I do not want to repeat.

6 Model deployment

6.1 What

Deployment here means exposing a trained model over HTTP so another service (or a human with curl) can request predictions without rerunning training. The FastAPI application (Ramírez 2026) in `Oli 2026o` serves `/health` and `/predict`. At startup it resolves a model URI: either a local `.joblib` path or `models:/lichess-outcome-predictor/Staging` from the MLflow registry.

6.2 Why

Serialising with `joblib` is simple; registering the same object in MLflow ties the running API to an experiment lineage. When April’s retrain beats March, I promote a new version to Staging and restart serving—no code change, only an environment variable. A network port (8080 by

default, 8082 for Prometheus scraping) satisfies the brief’s hint about models reachable beyond localhost, even though I have not yet wrapped the server in a Compose service.

6.3 How

`_load_model` runs in the FastAPI lifespan hook. It branches on whether `MODEL_URI` ends with `.joblib` or points at MLflow; the latter downloads the sklearn flavour through the tracking API. Request bodies follow [Oli 2026p](#): rating, title, time control, opening fields in `PredictRequest`, with class probabilities and a human-readable outcome label in `PredictResponse`. [Oli 2026k](#) supplies `build_inference_row` so training and serving share encoding logic—I fixed a nasty mismatch here once when one-hot columns diverged.

I start the server with `uv run lichess-serving` from the repo root after exporting `MODEL_URI` and MLflow tracking variables. The CLI in `packages/lichess_serving/src/lichess_serving/cli.py` wraps Uvicorn. Containerising is straightforward (slim Python image, same env vars) but left out of this submission to save integration time; the reflection section notes that gap honestly.

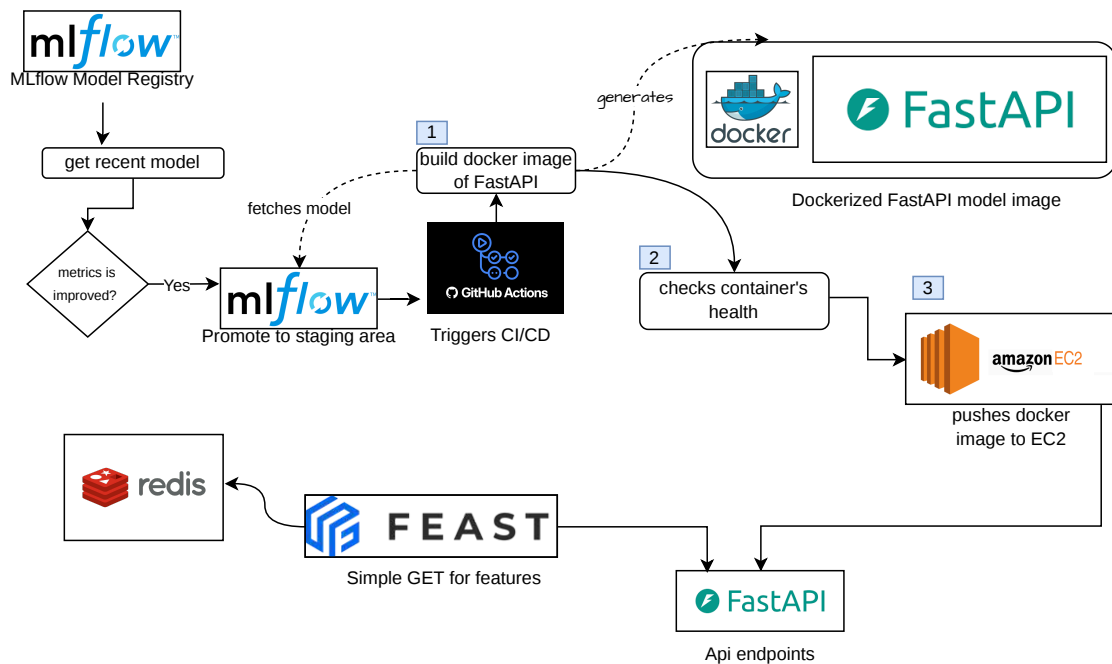


Figure 4: Deployment: promote model from MLflow, serve via FastAPI.

A minimal check I repeat after each promote: `GET /health` returns `loaded=true`, then `POST /predict` with a 1800-rated blitz game and the Sicilian Defence headers I used in tests. If probabilities sum to roughly one and the model gauge flips to 1, the deploy is good enough for demo purposes.

Serialisation and network access. The brief asks how models are stored and retrieved. On disk, `joblib` captures the fitted `Pipeline`; in MLflow, the same object is versioned with metrics and parameters. Over the network, JSON requests map to Python dicts, pass through the shared preprocessor, and return probabilities—no batch file hand-off. Docker would add an extra layer (image digest, published port 8080, health check in Compose) without changing that contract; I

documented the host-run path because it is what I demo today.

7 Model monitoring

7.1 What

Monitoring, in my setup, spans three layers: infrastructure metrics from the API, data-quality re-checks when a new month lands, and optional drift visualisation between reference and current feature files. Prometheus (Prometheus Authors 2026) scrapes the serving app; Grafana dashboards ship in the repo; Evidently (Evidently AI 2026) can render drift reports; Great Expectations reruns on fresh Parquet through  Oli 2026e.

7.2 Why

Accuracy on last month’s hold-out does not guarantee next month behaves. New openings surge after influencer games; rating inflation drifts feature means. Without `/metrics` I would not notice serving failures until a user complained. Without GE on the new shard I might train on a column that turned entirely null. Drift tooling is manual today, but it answers marker questions about “checking performance once deployed.”

7.3 How

`prometheus_fastapi_instrumentator` adds standard HTTP histograms. I defined `lichess_model_loaded` (gauge) and `lichess_predictions_total` (counter) in  Oli 2026o because they show up clearly on the dashboard JSON under  `services/prometheus/grafana/provisioning/dashboards/json/serving.json`. Compose profile monitoring starts Prometheus on 9090 and Grafana on 3000; the scrape config  Oli 2026v targets `host.docker.internal:8082` so a host-run server remains visible to containers.

After each monthly DAG run I would look at: (1) GE pass/fail in the artefact folder; (2) whether prediction rate spikes correlate with errors in `logs/lichess.log` via  Oli 2026q; (3) optionally POST reference and current Parquet paths to the Evidently API  (Oli 2026d) (`evidently` profile, port 5001) and open the HTML report in Streamlit on 8501. None of that is fully automated into Airflow yet—honestly, time boxed that integration.

Alerts in a production system would route GE failures and Airflow task errors to email or Slack; here, a failed task email from Airflow’s built-in notifier is enough to document intent. The monitoring stage completes the loop back to ingestion: a new month triggers download, validation, and the same observability hooks repeat.

8 Reflective discussion

Task 1 described a pipeline I had not yet wired: Airflow orchestration, MinIO layers, Spark transform, DuckDB analytics, Feast for temporal features, MLflow registry, FastAPI serving, and Prometheus plus Evidently for observability. The implementation in this repository follows that

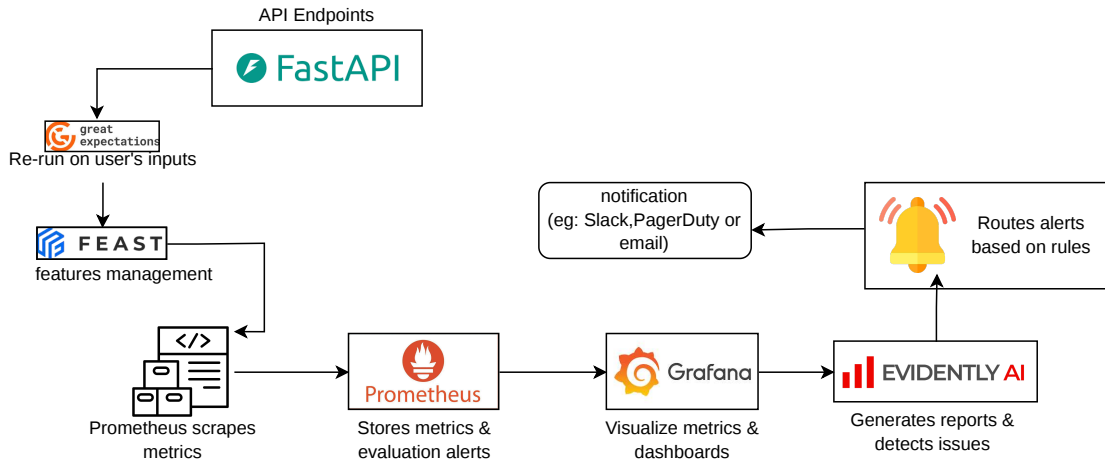


Figure 5: Monitoring: Prometheus, Grafana, drift reports, and pipeline alerts.

spine closely. Where the live system diverges, the reasons are mostly resource constraints on a student laptop rather than a change of mind about architecture.

Initial plan versus what I built. I planned ELT from the start and stuck with it. The surprise was how much shared plumbing (Oli 2026b, artefact paths, S3 helpers) mattered before any stage felt finished. I added the `use_elt` DAG flag after fighting Spark memory on a single-node cluster; the legacy extract path is not in the original diagram but saved weeks of blocked progress. Config and artefact managers from my early draft notes (`report2.txt`) became real modules instead of remaining bullet points.

Final plan versus gaps. Serving still runs on the host, not as a Compose service with health checks and rolling updates. Evidently drift is API-triggered: I copy Parquet into `services/evidently/data/` and call the report endpoint rather than a DAG task firing automatically. Retrain-on-drift is design-only. Great Expectations stores metadata in Postgres while validations execute from the host CLI—fine for demonstration, fragile for a team handover. These gaps are acceptable for coursework marks focused on “implemented pipeline with evidence,” but I would close them before calling the system production-ready.

What I would improve next. Containerise `lichess-serving` beside MLflow; add an Airflow branch after `validate_ge` that calls Evidently and opens an MLflow run when drift exceeds a threshold; promote models through Staging → Production with a canary metric gate; wire opening weakness analysis into the API so the player-facing story in my draft is user-visible, not a script. Storage-wise I would document retention on `lichess-raw` keys (months are large) and test restore from backup.

Comparing diagrams from Task 1 to this repo, the biggest lesson is iterative refinement the brief emphasises: the written plan was directionally right, but checksum gating, chronological splits, and shared config emerged from debugging real Lichess files, not from the first whiteboard pass.

Discrepancies worth naming. Task 1 showed decompression inside ingestion; I deferred decompress to Spark or the legacy extract precisely because ELT keeps the raw blob bit-identical to Lichess. Task 1 implied automated alerts to Slack; I rely on Airflow email and log files today.

Task 1’s monitoring diagram suggested continuous drift scoring; Evidently runs when I invoke it. None of these invalidate the architecture—they are scope cuts I would schedule if this were a team sprint.

Personal takeaway. Building the pipeline taught me more about data engineering than tuning the classifier. Chess data is messy, large, and interesting; tooling choices (MinIO, DuckDB, Airflow) only earn their keep when retrieval paths and validation gates are boringly reliable. I am satisfied the repository demonstrates implementation depth; the reflection is where I acknowledge what remains stubbed rather than pretending the last ten percent is done.

9 Conclusion

I implemented an end-to-end MLOps pipeline for the Lichess Open Database: monthly ingestion into MinIO with checksum gates, Spark and Pandas transformation into Parquet and DuckDB, Feast-backed feature splits validated by Great Expectations, sklearn training with MLflow tracking, FastAPI deployment from the model registry, and observability through Prometheus, Grafana, logs, and optional Evidently drift reports. Each stage maps to version-controlled code in LichessOps  (Oli 2026f), cited in this report and listed in Appendix A.

The work meets the module outcome I read as central to Task 2—appropriate structure for storage, retrieval, and analytical processing with justified software at each step. Raw games are stored as immutable objects, retrieved by URI for SQL and ML alike, and processed through tools chosen for columnar scale and temporal correctness. Limitations (host-based serving, manual drift) are documented openly in the reflection rather than hidden.

If I continued the project, automation would focus on closing the loop: new month in, validation pass, drift within bounds, retrain, promote, reload serving—all observable from one Grafana board. For now, the pipeline is a credible coursework implementation and a foundation I could demo without hand-waving the parts still stubbed.

A Code index

Table 1 lists major implementation files with GitHub links (). All paths are relative to repository root on branch `main`.

File	Description
 <code>libs/shared/config_loader.py</code>	YAML config merge
 <code>libs/shared/artifact_manager.py</code>	Artefact path resolution
<code>libs/shared/s3.py</code>	MinIO/S3 client helpers
 <code>libs/shared/storage_config.py</code>	DuckDB and S3 URI mapping
 <code>libs/shared/logger.py</code>	Rotating file logging
 <code>services/airflow/dags/lichess_monthly_ingestion.py</code>	Monthly orchestration DAG
 <code>packages/lichess_data/src/lichess_data/extract/lichess_downloader.py</code>	Shard download and checksum

File	Description
🔗 packages/lichess_data/src/lichess_data/validate/checks.py	SHA256 validation
🔗 packages/lichess_data/src/lichess_data/load/upload.py	Raw upload to MinIO
🔗 services/minio/docker-compose.yml	Object store and buckets
🔗 packages/lichess_data/src/lichess_data/spark/transform.py	Spark star-schema transform
🔗 packages/lichess_data/src/lichess_data/load/duckdb_sync.py	Parquet → DuckDB sync
🔗 packages/lichess_data/src/lichess_data/preprocessing/transforms.py	Pandas feature stages
🔗 packages/lichess_data/src/lichess_data/extract/pgn_parser.py	Legacy PGN parser
🔗 packages/lichess_features/src/lichess_features/split.py	Feast split and temporal cut
🔗 packages/lichess_data/src/lichess_data/validate/ge_runner.py	Great Expectations runner
🔗 packages/lichess_models/src/lichess_models/train.py	Hyperparameter search training
🔗 packages/lichess_models/src/lichess_models/features.py	Sklearn preprocessor
🔗 packages/lichess_models/src/lichess_models/dataset.py	Train/test loading
🔗 packages/lichess_models/src/lichess_models/evaluate.py	Hold-out metrics
🔗 packages/lichess_models/src/lichess_models/register.py	MLflow registration
🔗 services/mlflow/docker-compose.yml	MLflow tracking server
🔗 packages/lichess_serving/src/lichess_serving/app.py	FastAPI predict API
🔗 packages/lichess_serving/src/lichess_serving/schemas.py	API request/response models
🔗 services/prometheus/prometheus.yml	Metrics scrape config
🔗 services/evidently/src/app_fastapi/main.py	Drift report API

Table 1: Implementation code index with GitHub links.

References

- Apache Software Foundation (2026a). *Apache Airflow* URL: <https://airflow.apache.org/>.
- (2026b). *Apache Parquet* URL: <https://parquet.apache.org/>.
- Creative Commons (2025). *CC0 1.0 Universal — Public Domain Dedication* URL: <https://creativecommons.org/publicdomain/zero/1.0/>.
- Evidently AI (2026). *Evidently* URL: <https://www.evidentlyai.com/>.
- Feast (2026). *Feast — Feature Store for Machine Learning* URL: <https://feast.dev/>.
- Great Expectations (2026). *Great Expectations* URL: <https://greatexpectations.io/>.
- LF AI & Data Foundation (2026). *MLflow* URL: <https://mlflow.org/>.
- Lichess (2026). *lichess.org open database* URL: <https://database.lichess.org/>.
- Meta Platforms (2026). *Zstandard compression*  GitHub: <https://github.com/facebook/zstd>.
- MinIO, Inc. (2026). *MinIO — High Performance Object Storage* URL: <https://min.io/>.
- Oli, Nabin (2026a). *LichessOps: checks.py (checksum validation)*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/validate/checks.py.
- (2026b). *LichessOps: config_loader.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/libs/shared/config_loader.py.
- (2026c). *LichessOps: duckdb_sync.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/load/duckdb_sync.py.
- (2026d). *LichessOps: Evidently drift API*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/services/evidently/src/app_fastapi/main.py.
- (2026e). *LichessOps: ge_runner.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/validate/ge_runner.py.
- (2026f). *LichessOps: Lichess MLOps pipeline implementation. GitHub repository — nabin2004/LichessOps*  GitHub: <https://github.com/nabin2004/LichessOps>.
- (2026g). *LichessOps: lichess_downloader.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/extract/lichess_downloader.py.
- (2026h). *LichessOps: lichess_features/split.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_features/src/lichess_features/split.py.
- (2026i). *LichessOps: lichess_models/dataset.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/dataset.py.
- (2026j). *LichessOps: lichess_models/evaluate.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/evaluate.py.
- (2026k). *LichessOps: lichess_models/features.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/features.py.
- (2026l). *LichessOps: lichess_models/register.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/register.py.
- (2026m). *LichessOps: lichess_models/train.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/train.py.

- Oli, Nabin (2026n). *LichessOps: lichess_monthly_ingestion.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/services/airflow/dags/lichess_monthly_ingestion.py.
- (2026o). *LichessOps: lichess_serving/app.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_serving/src/lichess_serving/app.py.
 - (2026p). *LichessOps: lichess_serving/schemas.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_serving/src/lichess_serving/schemas.py.
 - (2026q). *LichessOps: logger.py*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/libs/shared/logger.py>.
 - (2026r). *LichessOps: MinIO docker-compose.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/minio/docker-compose.yml>.
 - (2026s). *LichessOps: MLflow docker-compose.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/mlflow/docker-compose.yml>.
 - (2026t). *LichessOps: pgn_parser.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/extract/pgn_parser.py.
 - (2026u). *LichessOps: preprocessing/transforms.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/preprocessing/transforms.py.
 - (2026v). *LichessOps: prometheus.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/prometheus/prometheus.yml>.
 - (2026w). *LichessOps: s3.py*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/libs/shared/s3.py>.
 - (2026x). *LichessOps: spark/transform.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/spark/transform.py.
 - (2026y). *LichessOps: storage_config.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/libs/shared/storage_config.py.
 - (2026z). *LichessOps: upload.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/load/upload.py.
- Prometheus Authors (2026). *Prometheus* URL: <https://prometheus.io/>.
- Ramírez, Sebastián (2026). *FastAPI* URL: <https://fastapi.tiangolo.com/>.
- scikit-learn developers (2026). *scikit-learn* URL: <https://scikit-learn.org/>.